

Actor-Critic Methods

Reinforcement Learning

Christian Igel

Department of Computer Science

🐦 @christian_igel

UNIVERSITY OF COPENHAGEN



Warm-up: Reparameterization trick

Let f be a deterministic function depending on a random variable $z \sim p_{\theta}$ parameterized by θ , we would like to know

$$\nabla_{\theta} \mathbb{E}[f(z)] \quad .$$

Because θ are the parameters of the random variable, it is not obvious how to apply a Monte Carlo method given a sample $z_1, \dots, z_n \sim p_{\theta}$.

Assume we can introduce a new random variable $\epsilon \sim q$ and rewrite

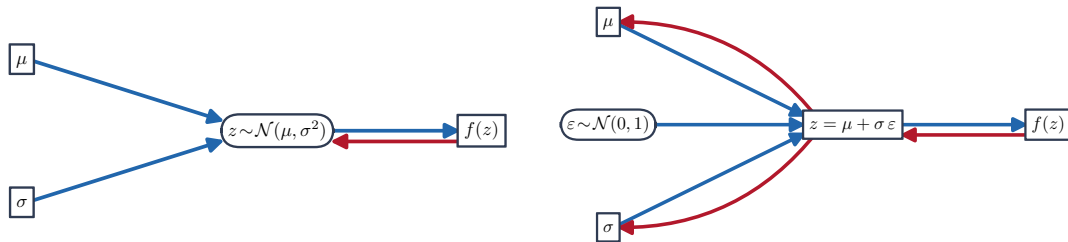
$$z = g_{\theta}(\epsilon) \text{ with } \epsilon \sim q$$

such that z when sampling ϵ still has the distribution p_{θ} .

Now we can estimate the gradient given an i.i.d. sample $\epsilon_1, \dots, \epsilon_n \sim q$:

$$\nabla_{\theta} \mathbb{E}_{z \sim p_{\theta}}[f(z)] = \nabla_{\theta} \mathbb{E}_{\epsilon \sim q}[f(g_{\theta}(\epsilon))] \approx \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} f(g_{\theta}(\epsilon_i))$$

Reparameterization trick example



Square and round nodes indicate deterministic and stochastic nodes, respectively. Blue arrows indicate the dependencies for computing the function f . Reverse-mode automatic differentiation follows the red arrows when computing the gradient. Left: Graph for a $f(z)$ with $z \sim \mathcal{N}(\mu, \sigma^2)$. Right: using reparameterization $z = g_{\theta}(\epsilon) = \mu + \sigma \epsilon$ with $\epsilon \sim \mathcal{N}(0, 1)$.

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Actors and critics

- A model approximating a value function is referred to as a critic.
- A model representing a policy mapping states to actions is called an actor.
- DQN is critic-only (mapping to actions does not have learnable parameters), needs to find the action that maximizes the value function in a given state $\rightarrow$ prohibitive for large action spaces.
- REINFORCE is actor-only and suffers from high variance of gradient estimates.
- Actor-critic algorithms combine both approaches.

Learning a baseline

The policy gradient theorem can be generalized, in either average-reward or start-state formulations, to include a baseline, e.g.,

$$\nabla_{\theta} J(\pi_{\theta}) = \sum_s \eta^{\pi_{\theta}}(s) \sum_a \nabla_{\theta} \pi_{\theta}(s, a) (Q^{\pi}(s, a) - b(s)) \quad ,$$

where $b(s) : S \rightarrow \mathbb{R}$ is an arbitrary baseline function.

$\nabla_{\theta} \pi_{\theta}(s, a) b(s)$ acts as a control variate. Note $\mathbb{E}[\nabla_{\theta} \pi_{\theta}(s, a) b(s)] = 0$.

A possible choice of $b(s)$ would be some estimate of the value function $V(s)$.

REINFORCE with baseline

Algorithm: REINFORCE with baseline

Input: policy π_{θ} , state-value function $\hat{V}$ parameterized by w , learning rates $\alpha_w, \alpha_{\theta} > 0$

1 **repeat**

2 $s_0 \sim p_{\text{start}}$

3 generate episode $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ following π

4 **foreach** $t = 0, \dots, T - 1$ **do**

5 $R_t = \sum_{k=1}^{T-t} \gamma^{k-1} r_{t+k}$

6 $w \leftarrow w + \alpha_w (R_t - \hat{V}(s_t)) \nabla_w \hat{V}(s_t)$ // critic update

7 $\theta \leftarrow \theta + \alpha_{\theta} \gamma^t (R_t - \hat{V}(s_t)) \nabla_{\theta} \ln \pi(s_t, a_t)$ // actor update

8 **until** *stopping criterion is met*

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Actor-Critic Methods

- Actor-critic method:
 - Policy π with parameters θ : actor
 - Value function with parameters w : critic
- REINFORCE with baseline is not fully online
- Introduce bootstrapping: Target not purely based on Monte Carlo return, but on previous estimate(s)
- Temporal difference learning: From

$$V^\pi(s_t) = \mathbb{E}^\pi [r_{t+1} + \gamma V^\pi(s_{t+1})]$$

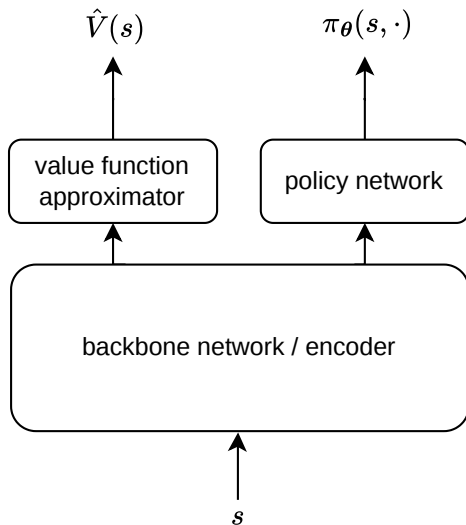
we get the halved squared TD error

$$\frac{1}{2}(\delta_{\text{TD}}^\pi(s, r, s'))^2 = \frac{1}{2} \left(\hat{V}(s) - \underbrace{(r + \gamma \hat{V}(s'))}_{\text{target } y} \right)^2$$

Recall notation

- A policy we currently follow/update is denoted by π_{θ} with parameters θ .
- A value function we learn is denoted by $\hat{V}$ or $\hat{Q}$ with parameters w .
- A policy that may be outdated and used to compute the target values for learning or serves as a reference is called π_{target} or π_{ref} .
- A value function that may be outdated and used to compute the targets for learning is denoted by $\hat{V}_{\text{target}}$ or $\hat{Q}_{\text{target}}$ with parameters w_{target} .

Two heads



Simple actor-critic algorithm

Algorithm: Simple actor-critic with TD learning

Input: policy π parameterized by θ , state-value function $\hat{V}$ parameterized by w , learning rates $\alpha_w, \alpha_\theta > 0$, initial θ and w

```
1 repeat
2    $t \leftarrow 0$ 
3    $s_t \sim p_{\text{start}}$ 
4   repeat
5      $a_t \sim \pi(s_t, \cdot)$ 
6     take action  $a_t$  and observe  $r_{t+1}$ ,  $s_{t+1}$ , and  $d_{t+1}$ 
7      $\delta \leftarrow r_{t+1} + (1 - d_{t+1})\gamma\hat{V}(s_{t+1}) - \hat{V}(s_t)$ 
8      $w \leftarrow w + \alpha_w \delta \nabla_w \hat{V}(s_t)$  // critic update
9      $\theta \leftarrow \theta + \alpha_\theta \gamma^t \delta \nabla_\theta \ln \pi(s_t, a_t)$  // actor update
10     $t \leftarrow t + 1$ 
11  until terminal state is reached
12 until stopping criterion is met
```

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Deep deterministic policy gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG) algorithm (Lillicrap et al., 2016):

- Model-free, off-policy actor critic
- Continuous action space
- Deterministic policy π_{θ}
- Random exploration modifying π_{θ} (e.g., additive zero mean Gaussian noise), easy in off-policy framework
- To stabilize learning with neural network:
 - Experience replay buffer
 - Target critic and target actor

Recall: Deterministic policy gradient

Corollary (deterministic policy gradient for discounted episodic tasks)

For any MDP with discount factor γ in the start-state setting and any deterministic policy π_{θ} it holds

$$\begin{aligned}\nabla_{\theta} J(\pi_{\theta}) &= \sum_s \eta_{\gamma}^{\pi_{\theta}}(s) \nabla_{\theta} \pi_{\theta}(s) \nabla_{\theta} Q^{\pi_{\theta}}(s, a) \big|_{a=\pi_{\theta}(s)} \\ &\propto \mathbb{E}^{\pi_{\theta}} \left[\sum_{k=0}^{\infty} \gamma^k \nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^{\pi_{\theta}}(s, a) \big|_{a=\pi_{\theta}(s)} \right] \\ &= \mathbb{E}^{\pi_{\theta}} \left[\sum_{k=0}^{\infty} \gamma^k \nabla_{\theta} Q^{\pi}(s, \pi_{\theta}(s)) \right] .\end{aligned}$$

DDPG updates

Mini-batch: $\mathcal{D} = \{\langle s_i, a_i, r_i, s'_i \rangle\}_{i=1}^{n_{\text{batch}}}$

Critic update: Gradient step descent following

$$\nabla_w \frac{1}{2N} \sum_{i=1}^N \underbrace{(r_i + \gamma \hat{Q}_{\text{target}}(s'_i, \pi_{\text{target}}(s'_i)) - \hat{Q}(s_i, a_i))}_{y_i}^2 =$$

$$- \frac{1}{N} \sum_{i=1}^N (y_i - \hat{Q}(s_i, a_i)) \nabla_{\theta} \hat{Q}(s_i, a_i)$$

Actor update: Gradient ascent following (ignoring the γ^t term):

$$\nabla_w \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \hat{Q}(s_i, \pi_{\theta}(s_i)) = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \nabla_{\theta} \pi_{\theta}(s_i) \nabla_a \hat{Q}(s_i, a) \big|_{a=\pi_{\theta}(s_i)}$$

Deep deterministic policy gradient (DDPG)

Algorithm: Deep Deterministic Policy Gradient

- 1 initial policy parameters θ , Q -function parameters w , experience buffer M
 - 2 $w_{\text{target}} \leftarrow w$, $\theta_{\text{target}} \leftarrow \theta$
 - 3 **repeat**
 - 4 observe s , choose a based on π (plus exploration)
 - 5 take action a , observe r, s'
 - 6 store $\langle s, a, r, s' \rangle$ in M
 - 7 sample mini-batch $\{\langle s_i, a_i, r_i, s'_i \rangle\}_{i=1}^{n_{\text{batch}}}$ from M
 - 8 set targets $y_i = r_i + \gamma \hat{Q}_{\text{target}}(s'_i, \pi_{\text{target}}(s'_i))$
 - 9 update $\hat{Q}$ network weights w
 - 10 update policy parameters θ
 - 11 update target networks:
$$\begin{aligned} w_{\text{target}} &\leftarrow \rho w_{\text{target}} + (1-\rho)w \\ \theta_{\text{target}} &\leftarrow \rho \theta_{\text{target}} + (1-\rho)\theta \end{aligned}$$
 - 12 **until** *until stopping criterion is met*
-

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Clipped double- Q -trick

- Overestimation can be a problem in off-policy methods
- The problem is amplified if function approximators are used, which introduce additional “noise” to the value estimate
- Actor-critic methods also claimed to suffer from overestimation bias, even if no explicit max operation is performed (e.g., see Fujimoto et al., 2018)
- Double Q -learning can be used to address this issue ($\rightarrow$ DDQN)
- Instead of switching between two Q functions with parameters w_1 and w_2 , one can use the “clipped double- Q -trick” (Fujimoto et al., 2018) and compute:

$$\min_{j=1,2} Q_{w_j}(s, a)$$

leading to an underestimation

Twin Delayed DDPG

- Twin Delayed DDPG (TD3) is an off-policy method, based on DDPG
- TD3 uses:
 - “clipped double- Q -trick”
 - experience replay buffer

Algorithm: Twin Delayed DDPG (TD3) w/o initialization

```

1 repeat
2   sample  $a = \text{clip}(\pi_{\theta}(s) + \epsilon, a_{\text{low}}, a_{\text{high}})$  with  $\epsilon \sim \mathcal{N}(0, \sigma_a^2)$ 
3   take action  $a$ , observe  $r, s', d$ 
4   store  $\langle s, a, r, s', d \rangle$  in  $M$ 
5    $s \leftarrow s'$ 
6   if  $d = 1$  then  $s \sim p_{\text{start}}$  // start of new episode
7   if  $T_{\text{update}}$  steps passed since last update then
8     for  $k = 1, \dots, n_{\text{gradient-steps}}$  gradient steps do
9       sample mini-batch  $\mathcal{D} = \{\langle s_i, a_i, r_i, s'_i, d_i \rangle\}_{i=1}^{n_{\text{batch}}}$  from  $M$ 
10      // compute targets
11      for  $i = 1, \dots, n_{\text{batch}}$  do
12         $\bar{a}_i = \text{clip}(\theta_{\text{target}}(s_i) + \epsilon_i, a_{\text{low}}, a_{\text{high}})$  with  $\epsilon_i \sim \mathcal{N}(0, \sigma_{\bar{a}}^2)$ 
13         $y_i = r_i + \gamma(1 - d_i) \min_{j=1,2} \hat{Q}_{\text{target},j}(s'_i, \bar{a}_i)$ 
14      // update critics
15      for  $j = 1, 2$  do
16        perform gradient-based minimization using:
17         $\nabla_{w_j} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \frac{1}{2} (\hat{Q}_j(s_i, a_i) - y_i)^2$ 
18      if  $k \bmod T_{\text{update-target}} = 0$  then // time for target update
19        perform gradient-based maximization using:
20         $\nabla_{\theta} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \hat{Q}_1(s'_i, \pi_{\theta}(s_i))$ 
21        for  $j = 1, 2$  do  $w_{\text{target},j} \leftarrow \rho w_{\text{target},j} + (1 - \rho) w_j$ 
22         $\theta_{\text{target}} \leftarrow \rho \theta + (1 - \rho) \theta_{\text{target}}$ 
23  until until stopping criterion is met
  
```

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Performance difference lemma

Lemma (performance difference lemma, discounted case)

For any two discounted policies π_{ref} and π we have:

$$J(\pi) = J(\pi_{\text{ref}}) + \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \underbrace{(Q^{\pi_{\text{ref}}}(s_t, a_t) - V^{\pi_{\text{ref}}}(s_t))}_{\text{advantage } A^{\pi_{\text{ref}}}(s, a)} \right]$$

When optimizing $J(\pi)$, we can maximize the advantages.

Advantage estimation I

We can estimate the advantage

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

from a sample s_t, a_t, r_{t+1} and approximated value function $\hat{V}^\pi$ by:

$$\hat{A}^\pi(s_t, a_t) = r_{t+1} + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)$$

Adding more steps incorporates more information and reduces the influence of a badly estimated $\hat{V}^\pi$. Starting from $\hat{A}_1^\pi(s_t, a_t) = r_{t+1} + \gamma \hat{V}^\pi(s_{t+1}) - \hat{V}^\pi(s_t)$ we have:

$$\begin{aligned} \hat{A}_k^\pi(s_t, a_t) &= \sum_{l=0}^{k-1} \gamma^l \hat{A}_1^\pi(s_{t+l}, a_{t+l}) = \\ &\quad -\hat{V}^\pi(s_t) + r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{k-1} r_{t+k} + \gamma^k \hat{V}^\pi(s_{t+k}) \end{aligned}$$

Advantage estimation II

Consider a (sub-)sequence $s_0, a_0, r_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots, s_T$ we can estimate the advantages:

$$\hat{A}_{T-t}^{\pi}(s_t, a_t) = -\hat{V}^{\pi}(s_t) + r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{T-t-1} r_T + \gamma^{T-t} \hat{V}^{\pi}(s_T)$$

In the limit we get the empirical return minus the value function baseline (which is just a control variate):

$$\hat{A}_{\infty}^{\pi}(s_t, a_t) = \sum_{l=0}^{\infty} \gamma^l \hat{A}_1^{\pi}(s_{t+l}, a_{t+l}) = -\hat{V}^{\pi}(s_t) + \sum_{l=1}^{\infty} \gamma^{l+1} r_{t+l}$$

It is a common strategy (PPO, ...) to break down episodes in subepisodes of length T for advantage computation.

(Asynchronous) Advantage Actor-Critic

- Asynchronous Advantage Actor-Critic = A³C
- Delayed critic update: $\hat{Q}$ and $\hat{Q}_{\text{target}}$ parameters w and w_{target}
- Initialization (not shown in algorithm on next page): initial actor parameters θ ; step counter $t \leftarrow 0$; initial value function weights $w = w_{\text{target}}$; accumulated gradients $\delta_w \leftarrow 0$, $\delta_\theta \leftarrow 0$; initial state $s \sim p_{\text{start}}$
- Algorithms are called asynchronous when designed for parallel threads which update the policy asynchronously. We present the single-thread “synchronous” version A²C.
- In our A²C version, T denotes the maximum number of time steps in a subepisode and we accumulated over T_{update} subepisodes, which could run in different threads.

Advantage Actor-Critic

Algorithm: A²C

```
1 repeat
2   for  $e = 1, \dots, T_{\text{update}}$  do
3      $t \leftarrow 0$ , observe  $s_t$ 
4     repeat
5       perform action  $a_t$  according to  $\pi(s_t, a_t)$ ; observe  $r_{t+1}, s_{t+1}, d_{t+1}$ 
6        $t \leftarrow t + 1$ 
7     until until  $d_t = 1$  or  $t = T$ 
8      $R \leftarrow (1 - d_t) \hat{V}_{\text{target}}(s_t)$ 
9     for  $i = t, \dots, 1$  do
10       $R \leftarrow r_i + \gamma R$  //  $i$  counts backwards
11       $\delta_{\theta} \leftarrow \delta_{\theta} + \nabla_{\theta} \log \pi(s_{i-1}, a_{i-1})(R - \hat{V}(s_{i-1}))$  //  $\rightarrow$  REINFORCE
12       $\delta_w \leftarrow \delta_w + \nabla_w (R - \hat{V}(s_{i-1}))^2$ 
13   update  $w$  and  $\theta$  using  $\delta_w$  and  $\delta_{\theta}$ 
14 until until stopping criterion is met
```

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Soft Actor Critic (SAC)

- SAC is a state-of-the-art off-policy algorithm (e.g., see T. Haarnoja et al., 2019) using a stochastic policy.
- SAC uses entropy regularization.
- SAC uses the “clipped double- Q -trick” to address overestimation.
- SAC applies “reparameterization trick” to compute gradient.
- SAC uses a delayed Q -function update

Entropy regularization I

- Problem (when using stochastic policies): Premature convergence to a (deterministic) suboptimal policy
- **Recall:** Entropy of a distribution is a measure of “randomness”/“surprise”. (the uniform distribution has maximum entropy; if p corresponds to a deterministic function, then $H(p)$ is lowest)

Shannon entropy $H(p) = \mathbb{E}_{x \sim p}[-\ln p(x)]$ of a discrete distribution p ;
differential entropy $H(p) = -\int p(x) \ln p(x) dx$ for continuous distributions p

- Reward for policies with large entropy (i.e., explore a lot):

$$\pi^* = \arg \max_{\pi} \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_{t+1} + \alpha H(\pi(s_t, \cdot)) \right) \right]$$

Entropy regularization II

Based on

$$\pi^* = \arg \max_{\pi} \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_{t+1} + \alpha H(\pi(s_t, \cdot)) \right) \right] ,$$

we define define:

$$\begin{aligned} V_H^{\pi}(s) &= \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_{t+1} + \alpha H(\pi(s_t, \cdot)) \right) \middle| s_0 = s \right] \\ Q_H^{\pi}(s, a) &= \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} + \alpha \sum_{t=1}^{\infty} \gamma^t H(\pi(s_t, \cdot)) \middle| s_0 = s, a_0 = a \right] \\ &= \mathbb{E}^{\pi} [r_{t+1} + \gamma Q_H^{\pi}(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a] \\ &\quad - \alpha_H \gamma \mathbb{E}^{\pi} [\log \pi(s_{t+1}, \cdot) \mid s_t = s] \end{aligned}$$

SAC value function update

Finite sample approximation

$$Q_H^\pi(s, a) \approx r_{t+1} + \gamma (Q_H^\pi(s', \tilde{a}') - \alpha \log \pi(s', \tilde{a}'))$$

with $\tilde{a}'$ being sampled anew and not being from the buffer:

$$\tilde{a}' \sim \pi(s', \cdot)$$

Error function for Q -networks ($i = 1, 2$) given data $\mathcal{D}$:

$$L(\mathbf{w}_i) = \mathbb{E}_{(s, a, r, s', d) \sim \mathcal{D}} \left[y(r, s', d) - \left(Q_{\mathbf{w}_i}(s, a) \right)^2 \right]$$

with target for $\tilde{a}' \sim \pi_\theta(s', \cdot)$:

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{j=1,2} Q_{\mathbf{w}_j^{\text{target}}}(s', \tilde{a}') - \alpha \log \pi_\theta(s', \tilde{a}') \right)$$

SAC policy update

SAC uses “reparameterization trick”.

Example: policy π_{θ} where continuous action in state s is Gaussian distributed with mean $\mu_{\theta}(s)$ and variance $\sigma_{\theta}(s)$ squashed by a sigmoid $\tanh$ function (avoiding extreme action values):

$$\tilde{a}_{\theta}(s, \xi) = \tanh(\mu_{\theta}(s) + \sigma_{\theta}(s) \odot \xi) \quad , \quad \xi \sim \mathcal{N}(0, \mathbf{I}) \Rightarrow$$

$$\begin{aligned} \mathbb{E}_{a \sim \pi_{\theta}(s, \cdot)} [Q^{\pi_{\theta}}(s, a) - \alpha \log \pi_{\theta}(s, a)] \\ &= \mathbb{E}_{a \sim \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s))} [Q^{\pi_{\theta}}(s, a) - \alpha \log \pi_{\theta}(s, a)] \\ &= \mathbb{E}_{\xi \sim \mathcal{N}(0, \mathbf{I})} [Q^{\pi_{\theta}}(s, \tilde{a}_{\theta}(s, \xi)) - \alpha \log \pi_{\theta}(\tilde{a}_{\theta}(s, \xi), s)] \end{aligned}$$

Putting it together ...

$$\nabla_{\theta} \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} \left(\underbrace{\min_{j=1,2} \hat{Q}_{\text{target},j}(s_i, \overbrace{\bar{\pi}_{\theta}(s_i, \xi_i)}^{\text{reparameterization trick}})}_{\text{clipped double } Q} - \underbrace{\alpha_H \log \overbrace{\bar{\pi}_{\theta}(s_i, \xi_i)}^{\text{reparameterization trick}}}_{\text{entropy regularization}} \right)$$

replaces $\hat{Q}(s_i, a_i)$ in DDPG

Soft Actor Critic algorithm (SAC)

Algorithm: Soft Actor Critic (SAC)

```

1 repeat
2   Observe  $s$ , select  $a \sim \pi_{\theta}$  and execute  $a$ 
3   Observe  $s'$ ,  $r$ , and terminal signal  $d$ ; store  $(s, a, r, s', d)$  in  $M$ 
4   if time to update then
5     for a number of gradient steps do
6       Randomly sample batch  $\mathcal{D}$  of transitions  $(s, a, r, s', d)$  from  $M$ 
7       Draw  $\tilde{a}'(s) \sim \pi_{\theta}(s, \cdot)$  using reparameterization trick
8       
$$y(r, s', d) = r + \gamma(1 - d) \left( \min_{i=1,2} Q_{\mathbf{w}_i^{\text{target}}}(s', \tilde{a}') - \alpha \log \pi_{\theta}(s', \tilde{a}') \right)$$

9       Perform updates for both  $i = 1, 2$  using:
10      
$$\nabla_{\mathbf{w}_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} \left( y(r, s', d) - Q_{\mathbf{w}_i}(s, a) \right)^2$$

11      
$$\nabla_{\theta} \frac{1}{|B|} \sum_{(s,\dots) \in B} \left( \min_{j=1,2} Q_{\mathbf{w}_j}(s, \tilde{a}_{\theta}(s)) - \alpha \log \pi_{\theta}(s, \tilde{a}_{\theta}(s)) \right)$$

12      
$$\mathbf{w}_i^{\text{target}} \leftarrow \rho \mathbf{w}_i^{\text{target}} + (1 - \rho) \mathbf{w}_i$$

13 until until stopping criterion is met

```

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Recall: Property of improper state distribution

For any function of states $g(s)$ and MPD with discount factor γ , we have

$$\sum_s \eta_{\gamma}^{\pi}(s) g(s) = \mathbb{E}^{\pi} \left[\sum_{t=0}^{\infty} \gamma^t g(s_t) \right] .$$

Proximal Policy Optimization

- Proximal Policy Optimization (PPO) is a popular deep RL method
- PPO uses importance weighting to become “more” on-policy
- PPO is rather robust, works for discrete and continuous action spaces
- PPO was used in training ChatGPT
- Ingredients:
 - “Surrogate loss” function CPI loss
 - Clipping
 - Optimize n_{steps} steps using mini-batches drawn from experience buffer
 - KL-Regularization

CPI loss function: Approximation I

The dependency on η_{γ}^{π} on the RHS makes

$$J(\pi) = J(\pi_{\text{ref}}) + \sum_s \eta_{\gamma}^{\pi}(s) \sum_a \pi_{\theta}(s, a) A^{\pi_{\text{ref}}}(s, a)$$

difficult to optimize. Thus, a local approximation is introduced:

$$J_{\pi_{\text{ref}}}^{\text{CPI}}(\pi) = \underbrace{J(\pi_{\text{ref}})}_{\substack{\text{can be dropped} \\ \text{when optimizing}}} + \sum_s \eta_{\gamma}^{\pi_{\text{ref}}}(s) \sum_a \pi(s, a) A^{\pi_{\text{ref}}}(s, a)$$

When optimizing $J_{\pi_{\text{ref}}}^{\text{CPI}}(\pi)$ w.r.t. to the parameters of π , the $J(\pi_{\text{ref}})$ term can be dropped as it is independent of π .

CPI loss function: Approximation II

Let θ be the parameter of the policy π . So far, our objective reads:

$$\arg \max_{\theta} \sum_s \eta_{\gamma}^{\pi_{\text{ref}}}(s) \sum_a \pi_{\theta}(s, a) A^{\pi_{\text{ref}}}(s, a)$$

Using $\sum_a \pi(s, a) A^{\pi_{\text{ref}}}(s, a) = \mathbb{E}^{\pi}[A^{\pi_{\text{ref}}}(s, a) | s]$ and dropping the γ^t term gives:

$$\arg \max_{\theta} \mathbb{E}^{\pi_{\text{ref}}} \left[\sum_{t=0}^{\infty} \mathbb{E}^{\pi_{\theta}} [A^{\pi_{\text{ref}}}(s_t, a) | s_t] \right]$$

Sample actions in a state s using π_{ref} as proposal distribution ($\rightarrow$ importance weighting) gives:

$$\arg \max_{\theta} \mathbb{E}^{\pi_{\text{ref}}} \left[\sum_{t=0}^{\infty} \frac{\pi_{\theta}(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)} A^{\pi_{\text{ref}}}(s_t, a_t) \right]$$

CPI loss function: Approximation III

We approximate based on one or several (sub-)episodes

$s_0, a_0, r_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots, s_T$ following π_{ref} .

This gives the CPI (named after *conservative policy iteration*, Kakade & Langford, 2002) objective

$$\sum_{t=0}^{T-1} \underbrace{\frac{\pi(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)}}_{\substack{\text{importance weighting} \\ \rightarrow \text{"more" on-policy}}} \hat{A}^{\pi_{\text{ref}}}(s_t, a_t)$$

to be maximized, see also advantage estimation on slide 24.

Recall: Clipping

- In the iterative optimization of a policy, it is a problem if a single step leads to a too big update, because
 - A single step may be a very noisy approximation of the proper gradient direction; and
 - A too large step may take the parameters outside the region where the first-order approximation underlying a gradient update is reasonable.
- To constrain the change of the policy, the update step (typically the approximated gradient) can be constrained, e.g., by limiting the length of the update vector or clipping the components of the update vector.
- PPO uses a very particular way of clipping the objective.

PPO Clipping

PPO-Clip objective:

$$\min \left(\frac{\pi(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)} \hat{A}^{\pi_{\text{ref}}}(s_t, a_t), \text{clip} \left(\frac{\pi(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}^{\pi_{\text{ref}}}(s_t, a_t) \right)$$

Clipping active $\rightarrow$ gradient of clipping term w.r.t. θ zero.

If $\hat{A}^{\pi_{\text{ref}}}(s_t, a_t) > 0$ optimization wants to increase $\pi(s_t, a_t)$ and $\min \left(\frac{\pi(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)}, 1 + \epsilon \right)$ limits increase if already $\frac{\pi(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)} > 1 + \epsilon$.

If $\hat{A}^{\pi_{\text{ref}}}(s_t, a_t) < 0$ optimization wants to decrease $\pi(s_t, a_t)$ and $\max \left(\frac{\pi(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)}, 1 - \epsilon \right)$ limits decrease if already $\frac{\pi(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)} < 1 - \epsilon$.

The policy is updated if $\frac{\pi(s_t, a_t)}{\pi_{\text{ref}}(s_t, a_t)} \in [1 - \epsilon, 1 + \epsilon]$ or if the gradient direction does not point away from that interval.

PPO

Algorithm: PPO

```
1 init policy and value function approximator parameters  $\theta$  and  $w$ 
2 repeat
3    $M = \emptyset$  // experience buffer
4   Gather experience // add samples  $\langle s_t, a_t, p_t, \hat{R}_t, \hat{A}_t \rangle$  to  $M$ 
5   for  $i = 1, \dots, n_{\text{steps}}$  do
6     Sample mini-batch  $\mathcal{D}$  from  $M$ 
7     Optimize PPO-Clip objective // update  $\theta$ 
8     Fit value function // update  $w$ 
9 until until stopping criterion is met
```

PPO sampling

Procedure Gather experience

```

1 for  $e = 1, \dots, T_{\text{update}}$  do
2    $t \leftarrow 0$ ; observe  $s_t$ 
3   repeat
4     perform action  $a_t \sim \pi(s_t, \cdot)$ ; observe  $r_{t+1}$  and  $s_{t+1}$ 
5      $p_t = \pi(s_t, a_t)$  // 'old' probabilities ( $\pi_{\text{ref}}(s_t, \cdot)$ )
6      $t \leftarrow t + 1$ 
7   until until  $t = T$ 
8   for  $t = 0, \dots, T - 1$  do
9      $\hat{R}_t(s_t, a_t) = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{T-t-1} r_T + \gamma^{T-t} \hat{V}(s_T)$ 
10     $\hat{A}_t(s_t, a_t) = \hat{R}_t(s_t, a_t) - \hat{V}(s_t)$ 
11  store  $\langle s_t, a_t, p_t, \hat{R}_t, \hat{A}_t \rangle$  in  $M$ 

```

We assume each subepisode runs for T steps. Policy and value function use parameters θ and w .

PPO-Clip optimization

Procedure Optimize PPO-Clip objective

- 1 Do gradient-based optimization following

$$\nabla_{\theta} \frac{1}{|\mathcal{D}|} \sum_{\langle s_t, a_t, p_t, \hat{R}_t, \hat{A}_t \rangle \in \mathcal{D}} \min \left(\frac{\pi(s_t, a_t)}{p_t} \hat{A}_t, \right. \\ \left. \text{clip} \left(\frac{\pi(s_t, a_t)}{p_t}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right)$$

where θ are the parameters of π

This is one step of the optimization for one mini-batch $\mathcal{D}$.

Fit value function

Procedure Optimize PPO-Clip objective

- 1 Do gradient-based optimization following

$$\nabla_w \frac{1}{|\mathcal{D}|} \sum_{\langle s_t, a_t, p_t, \hat{R}_t, \hat{A}_t \rangle \in \mathcal{D}} \left(\hat{R}_t - \hat{V}(s_t) \right)^2$$

where w are the parameters of $\hat{V}$

This is one step of the optimization for one mini-batch $\mathcal{D}$.

$\hat{V}$ and π may share parameters (parts of the network), then the gradients can be weighted, added and optimized jointly.

KL-Regularization I

- **KL-Regularization:** To prevent too large updates based on one batch that make learning unstable, the KL divergence

$$\text{KL}[\pi(s, \cdot) \parallel \pi_{\text{ref}}(s, \cdot)]$$

between current policy π and reference policy π_{ref} can be estimated over the observed states in a batch. It can serve as a constraint (Schulman et al., 2015) or can be added as a weighted penalty (to be minimized) to the objective $\rightarrow$ penalty for deviating from reference policy π_{ref} .

- **Entropy regularization** To ensure exploration (for discrete action spaces), the weighted entropy of the actions can be added to the objective (to be maximized). Can be viewed as KL-Regularization with uniform distribution as reference distribution.

KL-Regularization II

- For two distributions p and q , estimating

$$\text{KL}[q \parallel p] = \mathbb{E}_{x \sim q} \left[\log \frac{q(x)}{p(x)} \right] = \mathbb{E}_{x \sim q} \left[-\log \frac{p(x)}{q(x)} \right]$$

can have a high variance

- To reduce variance, we can introduce a (negatively correlated) control variate:

$$\int \frac{p(x)}{q(x)} - 1 \, \mathrm{d}q(x) = \int p(x) \, \mathrm{d}x - \int q(x) \, \mathrm{d}x = 0$$

giving:

$$\mathbb{E}_{x \sim q} \left[\log \frac{q(x)}{p(x)} + \frac{p(x)}{q(x)} - 1 \right] = \mathbb{E}_{x \sim q} \left[\frac{p(x)}{q(x)} - \log \frac{p(x)}{q(x)} - 1 \right]$$

GRPO

- Group Relative Policy Optimization (GRPO) is used in DeepSeekMath/DeepSeek (e.g., Shao et al., 2024) to train/tune large language models (LLMs).
- LLMs sample an answer given a question (prompt).
- GRPO does not use a learnt value function (i.e., it is actor-only), the target is computed as the average over a group of rewards returned in response to the same question.
- Advantages are computed relative to the group mean normalized by the standard deviation.
- Both PPO and GRPO use KL regularization, instead of adding the regularization to each reward, GRPO adds it as an additional loss term.

Deep RL summary: Stabilizing the learning

- Deep RL requires mechanisms to stabilize the learning. Influence of a single observed step and undesired “forgetting” is reduced by
 - Replay buffers ($\rightarrow$ sample efficiency),
 - Mini-batches,
 - Delayed Q and policy update ($\rightarrow$ averaging),
 - Clipping,
 - Regularization.
- Using different Q -functions as target mitigate. overestimation bias
- Advantage learning aims to reduce variance by introducing a covariate.

References II

C. Igel. *Concepts of Deep Reinforcement Learning: A Short Course*, chapter 6, 2026

S. Fujimoto, H. van Hoof, and D. Meger. Addressing Function Approximation Error in Actor-Critic Methods, International Conference on Machine Learning (ICML), 2018

T. Haarnoja et al. Learning to Walk via Deep Reinforcement Learning. Robotics: Science and Systems (RSS), 2019

S. Kakade, J. Langford. Approximately optimal approximate reinforcement learning. International Conference on Machine Learning (ICML), 2002

T. P. Lillicrap, et al.. Continuous control with deep reinforcement learning. International Conference on Learning Representations (ICLR), 2016

D. Silver, et al. Deterministic Policy Gradient Algorithms. ICML, 2014

References II

J. Schulman, et al. Trust region policy optimization. ICML, 2015

J. Schulman, et al. Proximal Policy Optimization Algorithms, 2017

J. Schulman, et al. High-dimensional continuous control using generalized advantage estimation. ICLR, 2016

Z. Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, 2024

Outline

REINFORCE with baseline

Actor-Critic Methods

Deep Deterministic Policy Gradient

Twin Delayed DDPG (TD3)

Asynchronous Advantage Actor-Critic

Soft Actor Critic

Proximal Policy Optimization

Summary

Deep RL: Prospects

Deep neural networks have led to tremendous progress in applied RL. They

- can learn highly non-linear functions, which allows them to model strong nonlinear interactions, represent multimodal functions, and approximate discontinuities;
- have proven to be applicable to large input spaces;
- can learn high-level representations from low-level sensory input;
- allow incorporation of prior knowledge in the network architecture (CNNs, RNNs, ...);
- allow iterative optimization using stochastic gradient descent.

Deep RL: Challenges

Training deep RL systems

- is resource intensive, needing a lot of data and compute;
- can be unstable;
- bears the risk of premature convergence to a suboptimal policy;
- involves many hyperparameters, which matter and are difficult to choose.

Algorithms we discussed

Algorithm	Actor	Critic	On-policy	Stochastic policy
CE / CMA-ES	✓	—	✓	—
DQN / DDQN	—	✓	—	—
REINFORCE	✓	—	✓	✓
REINFORCE w/ baseline	✓	✓	✓	✓
A ² C	✓	✓	✓	✓
DDPG	✓	✓	—	—
SAC	✓	✓	—	✓
TD3	✓	✓	—	—
PPO	✓	✓	(✓)	✓
GPPO	✓	—	(✓)	✓

And in practice?

- Best deep RL algorithm for a task is highly dependent on the task and the available resources.
- Performance crucially depends on hyperparameters.
- For specialized domains, specialized algorithms may exist that can be expected to outperform general approaches.
- PPO: rather robust and has worked well in practice for many applications.
- SAC is recommended for continuous control.
- For discrete action spaces, variants of DDQN are not outdated.
- For policies with only a few parameters, CMA-ES is an easy to use option.
- Stable Baselines3 provides clean PyTorch implementations.